

D e c o r a t o r

# Decorator

## Pattern

Add behavior by wrapping

### DEFINITION

Attach responsibilities to an object dynamically by wrapping it in objects of the same interface.

Decorator wraps an object in a layer that implements the same interface and adds behavior before or after delegating. Because the wrapper is type-compatible with what it wraps, you stack features in any order at runtime, transparently to clients — the composition answer to subclass explosion.

### WHY IT MATTERS

When optional features combine in many ways, a subclass per combination explodes (a `BufferedEncryptedLoggingStream`, and so on). Decorator keeps one interface and layers behavior with composition, so you start with a plain object and add one wrapper at a time, mixing features freely. The costs: many small look-alike objects, broken identity (a decorator is not `==` its component), and order-sensitive stacks.



# — How it works

**Component**                      The shared interface real objects and wrappers both implement.

**Decorator**                      Abstract wrapper — holds a Component reference and delegates to it.

**ConcreteDecorator** Adds state or behavior before / after delegating the call.

HOW TO APPLY

- 1. Define the Component interface the base object and wrappers share.
- 2. Give each decorator a Component reference and delegate the core call to it.
- 3. Add behavior before or after the delegation in each ConcreteDecorator.
- 4. Stack wrappers at runtime; make the order explicit, it matters.

LITMUS TEST

Do optional features combine in many ways, and must they attach to individual objects at runtime? If subclassing every combination would explode, wrap instead.



## — Smell → fix

A subclass for every feature combination explodes:

```
class Coffee {}  
class MilkCoffee extends Coffee {}  
class SugarMilkCoffee extends MilkCoffee {} // and on  
// every feature combo needs its own subclass
```

↓ WRAP INSTEAD OF SUBCLASS

```
interface Coffee { cost(): number }  
const espresso = (): Coffee => ({ cost: () => 2 })  
const milk = (c: Coffee): Coffee => ({  
  cost: () => c.cost() + 0.5 // wrap, then add  
})  
milk(espresso()).cost() // 2.5 — stacked at runtime
```

milk() wraps any Coffee and adds to cost(); stack milk(sugar(espresso())) in any order, with no new class per combo. Mind that wrapping breaks identity (a decorator is not == its component) and the stack order changes behavior.

### USE WHEN

- ✓ Optional features combined at runtime
- ✓ Avoid a subclass explosion from feature combos

### AVOID WHEN

- ▲ Deep wrapper stacks become hard to trace; order-sensitive



## — Trade-offs & pitfalls

### PROS

- + Composition over inheritance
- + Mix features freely

### CONS

- Many small objects
- Wrapping order matters

### COMMON PITFALLS

- ▲ Order matters: `retry(cache(x))` differs from `cache(retry(x))` — make the stack explicit.
- ▲ Wrapping breaks identity: `instanceof` and `==` against the inner object fail; unwrap when you must.
- ▲ Not the TS `@decorator`: GoF wraps instances at runtime; TS / Angular decorators annotate at declaration.

### WHERE YOU SEE IT

- ▶ `java.io` streams (`BufferedInputStream` wraps `FileInputStream`); Node Transform streams (`gzip`, `cipher`).
- ▶ NestJS interceptors and middleware pipelines wrapping a handler with caching, logging, timeout.
- ▶ Function decorators / HOFs (Python `@lru_cache`, React HOCs, RxJS operators).

### SEE ALSO

#### Proxy

same wrapping shape — but controls access (lazy, guard) rather than adding behavior

#### Adapter

also wraps one object — but changes the interface instead of keeping it

#### Composite

a Decorator is a Composite with a single child that adds behavior

#### Col

"favor composition over inheritance" — the principle Decorator embodies



## — Interview & sources

### Decorator vs Proxy?

Same wrapping shape, different intent — Decorator adds behavior, Proxy controls access (lazy, remote, guard).

### Decorator vs inheritance?

Inheritance fixes behavior at compile time; decorators compose it at runtime and dodge subclass explosion.

### Is a TS @decorator the GoF pattern?

No — TS / Angular decorators are declaration-time metadata, not runtime instance wrappers.

### The main downside?

Many small look-alike objects, broken identity checks, and order-sensitive stacks.

#### REMEMBER

Wrap an object in a same-typed layer that adds behavior — compose features at runtime, no subclass explosion.

#### SOURCES

GoF — "Design Patterns" (1994): Decorator (structural)

Freeman & Robson — "Head First Design Patterns" (2nd ed., 2020): Decorator